

Approches classiques de l'IA

Chapitre 8 : Apprentissage automatique supervisé

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 8. Apprentissage automatique supervisé

- 1 Apprentissage automatique
- 2 Apprentissage supervisé
- 3 Arbres de décision
- 4 Evaluer et choisir la meilleure hypothèse
- 5 Introduction aux IA génératives
- 6 Résumé du chapitre

Apprentissage automatique

Apprentissage automatique (Machine Learning)

Dans ce chapitre on introduit (très brièvement) le domaine de l'apprentissage automatique (Machine Learning – ML).

Discipline à part entière se basant sur divers domaines.
Principalement :

- statistiques ;
- probabilités ;
- intelligence artificielle ;
- recherche opérationnelle ;
- théorie de l'information ¹.

1. Grossièrement : théorie probabiliste pour quantifier le contenu informatif de messages.

Apprentissage automatique (Machine Learning)

Un agent **apprend** s'il améliore ses performances futures à partir de ce qu'il observe.

Apprentissage automatique (Machine Learning)

Un agent **apprend** s'il améliore ses performances futures à partir de ce qu'il observe.

Dans ce chapitre : une classe spécifique de problèmes d'apprentissage.

Problème

A partir d'une collection de paires (*entrée, sortie*) **apprendre** une fonction qui **prédit** les sorties adéquates pour de **nouvelles** entrées.

Peut sembler restrictif mais s'applique à une grande variété de problèmes d'apprentissage.

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Car les concepteurs d'agents intelligents...

- ne peuvent pas anticiper toutes les situations possibles ;

Exemple

Robot qui se déplace dans des labyrinthes ou environnements dont la "géographie" est inconnue.

- ne peuvent pas anticiper les changements futurs ;

Exemple

Un programme pour prédire les prix de la bourse du lendemain doit s'adapter à la conjoncture économique (récession ou "boom").

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Car les concepteurs d'agents intelligents...

- ne peuvent pas anticiper toutes les situations possibles ;

Exemple

Robot qui se déplace dans des labyrinthes ou environnements dont la "géographie" est inconnue.

- ne peuvent pas anticiper les changements futurs ;

Exemple

Un programme pour prédire les prix de la bourse du lendemain doit s'adapter à la conjoncture économique (récession ou "boom").

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Car les concepteurs d'agents intelligents...

- ne peuvent pas anticiper toutes les situations possibles ;

Exemple

Robot qui se déplace dans des labyrinthes ou environnements dont la "géographie" est inconnue.

- ne peuvent pas anticiper les changements futurs ;

Exemple

Un programme pour prédire les prix de la bourse du lendemain doit s'adapter à la conjoncture économique (récession ou "boom").

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Il y a une troisième raison importante due à la nature de certains problèmes. . .

- les concepteurs n'ont, parfois, aucune idée de comment programmer une solution.

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Il y a une troisième raison importante due à la nature de certains problèmes. . .

- les concepteurs n'ont, parfois, aucune idée de comment programmer une solution.

Apprentissage automatique (Machine Learning)

Pourquoi un agent doit-il apprendre ? Il y a une troisième raison importante due à la nature de certains problèmes. . .

- les concepteurs n'ont, parfois, aucune idée de comment programmer une solution.

Exemple

La plupart des gens sont capables de reconnaître les visages de membres d'une même famille. Même les meilleurs programmeurs sont incapables d'écrire un programme pour réaliser cette tâche en utilisant de la programmation classique.

Apprentissage automatique (Machine Learning)

Exemples d'applications

- lutte contre le spam
- moteurs de recherches
- diagnostique médical
- traitement du langage naturel
- traduction automatique
- analyse économique
- robotique (e.g., apprentissage des mouvements)
- génie logiciel
- bioinformatique
- classification de séquences ADN
- reconnaissance vocale
- reconnaissance manuscrite (e.g., signature)
- reconnaissance d'objets (computer vision)
- etc.

Apprentissage supervisé

Formes d'apprentissage

Trois types d'apprentissage principaux en fonction du feedback :

- L'apprentissage **supervisé** : l'agent observe des paires (*entrée, sortie*) et apprend une fonction qui associe les entrées aux sorties $\Rightarrow$ nécessite un “**enseignant**”
- L'apprentissage par **renforcement** : l'agent apprend à partir d'une série d'expériences pour lesquelles il reçoit une indication positive ou négative $\Rightarrow$ **pas besoin** d'enseignant
- L'apprentissage **non supervisé** : l'agent apprend des motifs dans les exemples donnés en entrée sans recevoir un feedback spécifique (pas de “sorties” données) $\Rightarrow$ Typiquement : **clustering**, **pas besoin** d'enseignant

Formes d'apprentissage

En pratique, la distinction n'est pas toujours si claire : souvent, apprentissage **semi-supervisé** :

- quelques exemples “étiquetés” (pour lesquels on a du feedback) et beaucoup d'exemples “non-étiquetés”.
- les étiquettes elles-mêmes peuvent être erronées (bruit ou absence des données, mesures imprécises, etc. : **inévitable** en pratique).

Apprentissage supervisé

Dans ce chapitre, on présente uniquement l'apprentissage supervisé avec les hypothèses suivantes :

- on ignore la connaissance initiale ;
- on suppose que les exemples sont **donnés** (et correctement étiquetés) ;
- on suppose que l'agent veut approximer une **fonction cible f** .

Apprentissage supervisé

Soit f une fonction cible (inconnue).

Un exemple est une paire $(x, f(x))$.

Apprentissage supervisé

Soit f une fonction cible (inconnue).

Un exemple est une paire $(x, f(x))$.

Remarque : x et $f(x)$ ne doivent pas forcément être des nombres.

Exemple (Jeu d'OXO)

$$x = \begin{bmatrix} O & O & X \\ \hline & X & \\ \hline X & & \end{bmatrix}, \quad f(x) = +1$$

Apprentissage supervisé

Apprendre une fonction à partir d'exemples

Problème : trouver une fonction h (l'hypothèse) telle que $h \simeq f$, à partir d'un ensemble d'entraînement de N paires d'exemples (training set) :

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N),$$

où $y_j = f(x_j)$ pour tout j .

Apprentissage supervisé

Apprendre une fonction à partir d'exemples

Problème : trouver une fonction h (l'hypothèse) telle que $h \simeq f$, à partir d'un ensemble d'entraînement de N paires d'exemples (training set) :

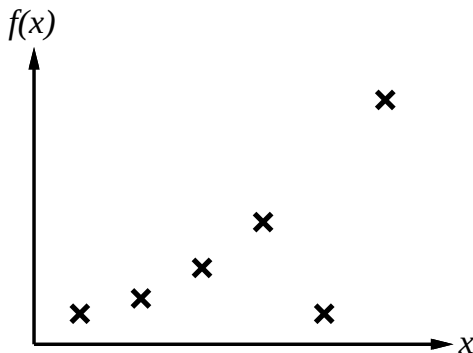
$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N),$$

où $y_j = f(x_j)$ pour tout j .

Pour mesurer la précision de l'hypothèse h , on utilisera un ensemble de test (test set) d'exemples (différents de ceux du training set).

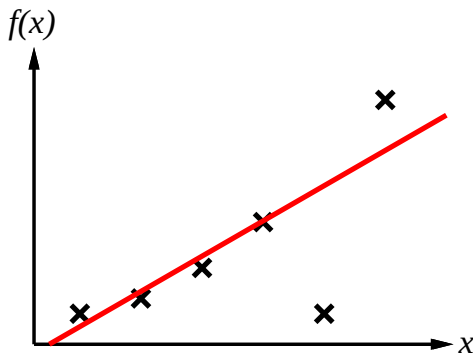
Apprentissage supervisé

Construire / ajuster h pour généraliser f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



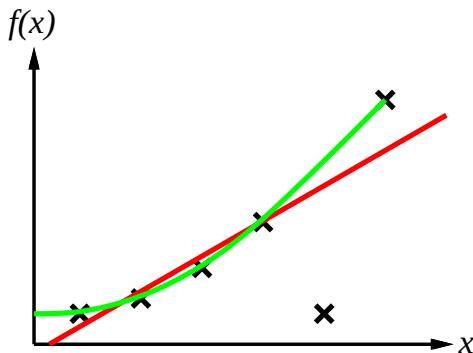
Apprentissage supervisé

Construire / ajuster h pour généraliser f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



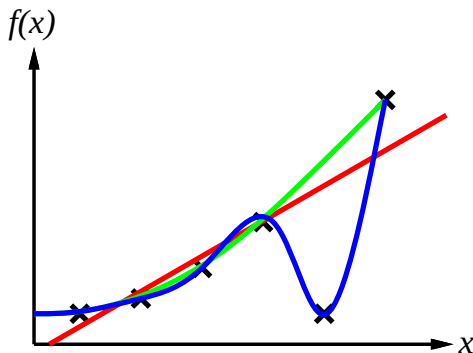
Apprentissage supervisé

Construire / ajuster h pour généraliser f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



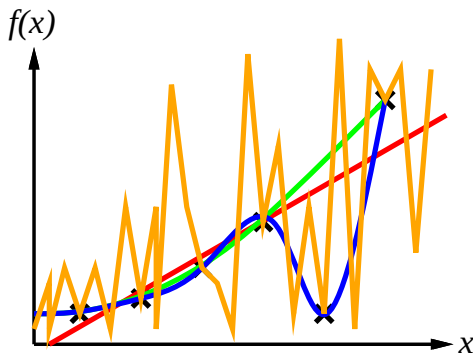
Apprentissage supervisé

Construire / ajuster h pour généraliser f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



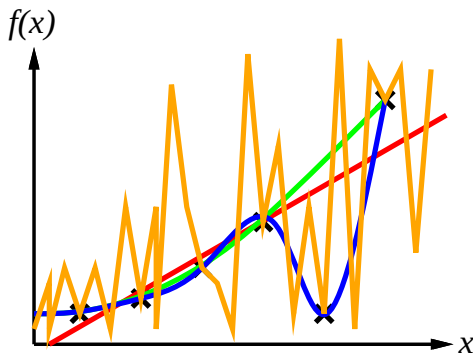
Apprentissage supervisé

Construire / ajuster h pour approximer f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



Apprentissage supervisé

Construire / ajuster h pour approximer f sur le training set (h est **consistante** si $h(x) = f(x)$ pour tous les exemples donnés)



Principe du **rasoir d'Ockham** : choisir l'hypothèse consistante la plus simple (la **simplicité** étant à définir)

Apprentissage supervisé

Remarques :

- On dit qu'une hypothèse **généralise** bien f si elle prédit correctement la valeur de y pour des nouveaux exemples.
- Quand la sortie y est une valeur parmi un ensemble fini de valeurs discrètes, le problème d'apprentissage est appelé une **classification**.
- Quand y est un nombre (comme la température du lendemain), le problème est appelé une **régression**.
- Une hypothèse h est choisie dans un **espace d'hypothèses** $\mathcal{H}$ (par ex. l'ensembles des polynômes).

Apprentissage supervisé

Remarques :

- On dit qu'une hypothèse **généralise** bien f si elle prédit correctement la valeur de y pour des nouveaux exemples.
- Quand la sortie y est une valeur parmi un ensemble fini de valeurs discrètes, le problème d'apprentissage est appelé une **classification**.
- Quand y est un nombre (comme la température du lendemain), le problème est appelé une **régression**.
- Une hypothèse h est choisie dans un **espace d'hypothèses** $\mathcal{H}$ (par ex. l'ensembles des polynômes).

Apprentissage supervisé

Remarques :

- On dit qu'une hypothèse **généralise** bien f si elle prédit correctement la valeur de y pour des nouveaux exemples.
- Quand la sortie y est une valeur parmi un ensemble fini de valeurs discrètes, le problème d'apprentissage est appelé une **classification**.
- Quand y est un nombre (comme la température du lendemain), le problème est appelé une **régression**.
- Une hypothèse h est choisie dans un **espace d'hypothèses** $\mathcal{H}$ (par ex. l'ensembles des polynômes).

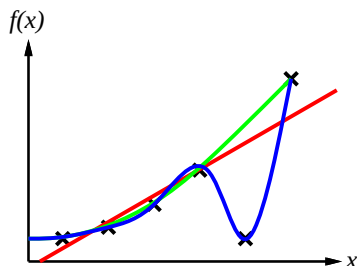
Apprentissage supervisé

Remarques :

- On dit qu'une hypothèse **généralise** bien f si elle prédit correctement la valeur de y pour des nouveaux exemples.
- Quand la sortie y est une valeur parmi un ensemble fini de valeurs discrètes, le problème d'apprentissage est appelé une **classification**.
- Quand y est un nombre (comme la température du lendemain), le problème est appelé une **régression**.
- Une hypothèse h est choisie dans un **espace d'hypothèses** $\mathcal{H}$ (par ex. l'ensembles des polynômes).

Apprentissage supervisé

Exemple : espace d'hypothèses = ensembles des polynômes.



Polynôme de degré 1 (droite rouge) ; polynôme de degré 2 (courbe quadratique verte) ; etc.

Principe du rasoir d'Ockham : choisir le polynôme consistant du plus petit degré.

Apprentissage supervisé

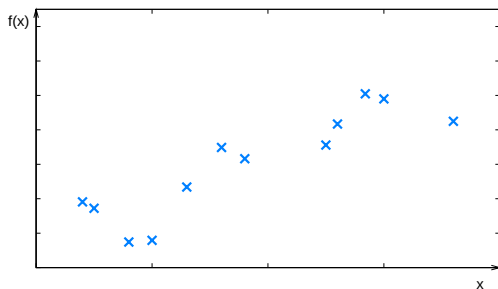
Trouver une bonne hypothèse h est une histoire de compromis

Hypothèse complexe qui correspond exactement aux données d'entraînement ou hypothèse plus simple qui pourrait mieux généraliser la fonction cible ?

Apprentissage supervisé

Trouver une bonne hypothèse h est une histoire de compromis

Hypothèse complexe qui correspond exactement aux données d'entraînement ou hypothèse plus simple qui pourrait mieux généraliser la fonction cible ?

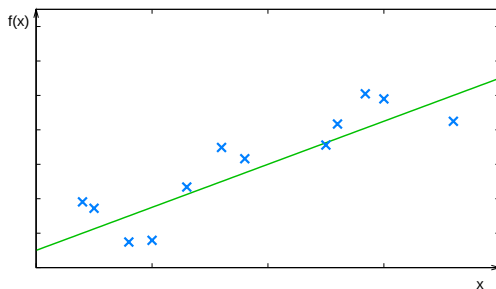


Si en réalité f n'est pas un polynôme et que $\mathcal{H}$ = ensemble des polynômes, alors le degré de h sera proche du nombre d'exemples !

Apprentissage supervisé

Trouver une bonne hypothèse h est une histoire de compromis

Hypothèse complexe qui correspond exactement aux données d'entraînement ou hypothèse plus simple qui pourrait mieux généraliser la fonction cible ?

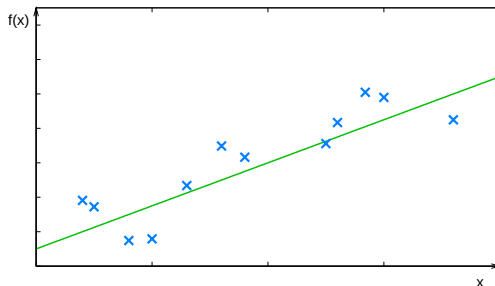


Alternative à un polynôme de haut degré : choisir une fonction plus simple dans $\mathcal{H}$, qui n'est pas consistante mais qui approche bien f .

Apprentissage supervisé

Trouver une bonne hypothèse h est une histoire de compromis

On dit qu'un problème d'apprentissage est **réalisable** si l'espace d'hypothèse contient la véritable fonction cible.



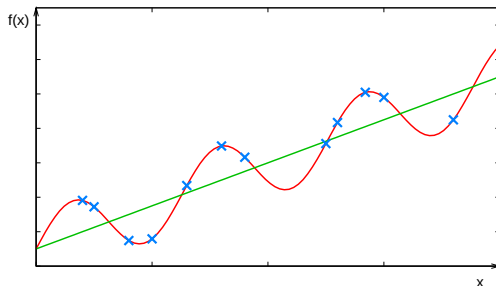
Apprentissage supervisé

Trouver une bonne hypothèse h est une histoire de compromis

On dit qu'un problème d'apprentissage est **réalisable** si l'espace d'hypothèse contient la véritable fonction cible.

Dans l'exemple ci-dessous,

$$f(x) = \frac{x}{2} + 2\sin(x) + 1.$$



Apprentissage supervisé

Trouver une bonne hypothèse h est une histoire de compromis

Solution à l'exemple précédent : définir $\mathcal{H}$ comme l'ensemble des polynômes sur x et sur $\sin(x)$.

→ compromis entre l'expressivité de l'espace des hypothèses et la complexité de trouver une bonne hypothèse dans cet espace.

Arbres de décision

Représentation par arbres de décision

Un **arbre de décision** représente une **fonction** qui

- prend en entrée un vecteur de valeurs d'attributs (représentation **factorisée**),
- retourne une “décision” : une seule valeur parmi un ensemble fini (**classification**),
- utilise une structure arborescente où un nœud = un **test sur la valeur d'un des attributs**.

Pour illustrer, on se contente d'une **classification booléenne** : chaque exemple sera classé comme vrai (un exemple **positif**) ou faux (un exemple **négatif**).

→ entrée x = vecteur d'attributs, sortie y = booléen

Représentation par arbres de décision

Exemple : on va construire un arbre de décision pour décider s'il faut attendre pour une table à un restaurant.

But : apprendre une définition du **prédicat objectif** *WillWait*.

Attributs (d'une entrée au problème) :

- *Alternate* : y a-t-il une alternative valable proche ?
- *Bar* : y a-t-il un bar ?
- *Fri/Sat* : vrai les vendredis et samedis.
- *Hungry* : vrai si on a faim.
- *Patrons* : (= clients) combien y a-t-il de personnes dans le restaurant (valeurs possibles : *None*, *Some* ou *Full*).
- ... (suite slide suivant)

Représentation par arbres de décision

Attributs (suite) :

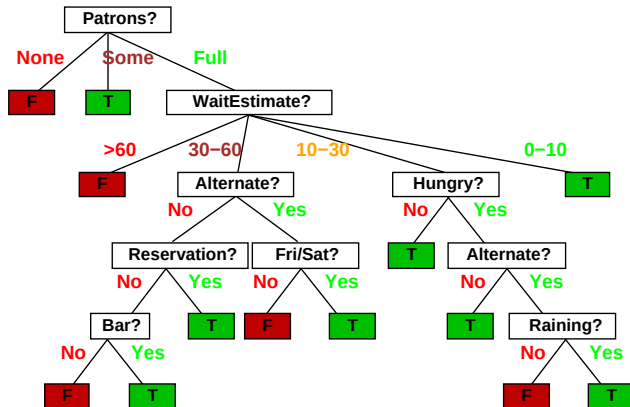
- *Price* : le prix moyen du restaurant (valeurs possibles : \$, \$\$ ou \$\$\$).
- *Raining* : vrai s'il pleut.
- *Reservation* : vrai si on a une réservation.
- *Type* : type de restaurant (parmi *French*, *Italian*, *Thai* ou *Burger*).
- *WaitEstimate* : le temps d'attente estimé (0 – 10 min., 10 – 30, 30 – 60 ou > 60).

Arbre de décision :

- nœud interne = test sur la valeur d'un des attributs ci-dessus
- feuille = vrai ssi on attend pour une table en fonction des réponses aux tests suivis depuis la racine jusqu'à la feuille

Représentation par arbres de décision

Un représentation possible pour la fonction cible f (pour une personne donnée, ici S. Russel) :



Apprendre un arbre de décision à partir d'exemples

Soit le training set suivant :

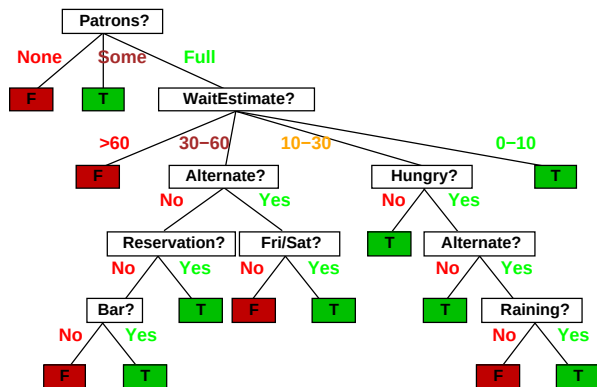
Ex.	Attributes									
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>
x_1	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>0–10</i>
x_2	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>30–60</i>
x_3	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>Some</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>0–10</i>
x_4	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>10–30</i>
x_5	<i>T</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>French</i>	<i>>60</i>
x_6	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Italian</i>	<i>0–10</i>
x_7	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>0–10</i>
x_8	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>Some</i>	<i>\$\$</i>	<i>T</i>	<i>T</i>	<i>Thai</i>	<i>0–10</i>
x_9	<i>F</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>Full</i>	<i>\$</i>	<i>T</i>	<i>F</i>	<i>Burger</i>	<i>>60</i>
x_{10}	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$\$\$</i>	<i>F</i>	<i>T</i>	<i>Italian</i>	<i>10–30</i>
x_{11}	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>None</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Thai</i>	<i>0–10</i>
x_{12}	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>Full</i>	<i>\$</i>	<i>F</i>	<i>F</i>	<i>Burger</i>	<i>30–60</i>

Apprendre un arbre de décision à partir d'exemples

Exercice

Est-ce que S. Russel attendrait une table dans les conditions décrites par x_2 et x_3 ?

$f =$



Apprendre un arbre de décision

Exercice

Est-ce que S. Russel attendrait une table dans les conditions décrites par x_2 et x_3 ?

Réponse. Valeurs de *WillWait* : $y_2 = f(x_2) = \text{No}$ et $y_3 = f(x_3) = \text{Yes}$.

Apprendre un arbre de décision

Exercice

Est-ce que S. Russel attendrait une table dans les conditions décrites par x_2 et x_3 ?

Réponse. Valeurs de *WillWait* : $y_2 = f(x_2) = \text{No}$ et $y_3 = f(x_3) = \text{Yes}$.

Sur les 12 exemples, on obtient 6 exemples **positifs** :

$$\{x_1, x_3, x_4, x_6, x_8, x_{12}\},$$

et 6 exemples **négatifs** :

$$\{x_2, x_5, x_7, x_9, x_{10}, x_{11}\}.$$

Apprendre un arbre de décision

Problème de l'apprentissage supervisé : on ne reçoit que les réponses (y_j) et on ne connaît pas f , comment déterminer un bon arbre h ?

Apprendre un arbre de décision

Problème de l'apprentissage supervisé : on ne reçoit que les réponses (y_j) et on ne connaît pas f , comment déterminer un bon arbre h ?

On va chercher un arbre de décision **consistant** avec les exemples du training set et si possible **pas trop grand**.

→ Algorithme DTL (Decision Tree Learning)

Algorithme DTL (Decision tree learning)

Algorithme DTL

- **But** : trouver un petit arbre de décision consistant avec les exemples
- **Idée** : (récursivement) choisir l'attribut le “plus significatif” comme racine d'un (sous)-arbre (stratégie “greedy divide-and-conquer”²)
- **Fonctions** annexes :
 - ▶ **PLURALITY**(*exas*) : retourne la réponse (y_j) la plus fréquente pour un ensemble d'exemples *exas* (plurality = majorité relative). Bris d'égalité aléatoire : **DTL n'est pas déterministe**.
 - ▶ **IMPORTANCE**(*a*, *exas*) : retourne une valeur exprimant l'importance d'un attribut *a* donné parmi un ensemble d'exemples *exas*. Plus cette valeur est grande, plus l'attribut est “important”.

2. Trouver le plus petit arbre est NP-difficile (algo naïf = parcourir les 2^{2^n} arbres), d'où l'utilisation d'une heuristique rapide.

Algorithme DTL (Decision tree learning)

```

function DTL(examples, attributes, parent_exs) returns a dec. tree

  if examples is empty then return PLURALITY(parent_exs)
  else if all examples have the same classification
    then return the classification
  else if attributes is empty then return PLURALITY(examples)
  else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    tree  $\leftarrow$  a new decision tree with root test A
    for each value  $v_k$  of A do
       $\text{exs} \leftarrow \{e \in \text{examples} \text{ and } e.A = v_k\}$ 
      subtree  $\leftarrow$  DTL(exs, attributes – A, examples)
      add a branch to tree with label (A =  $v_k$ ) and subtree subtree
    return tree
  
```

Choisir un attribut – IMPORTANCE

Idée : un “bon” attribut **sépare** les exemples en sous-ensembles qui peuvent (idéalement) être tous classés de la même façon.

Exercice

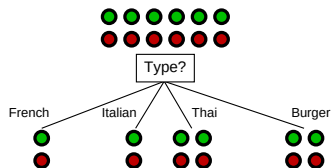
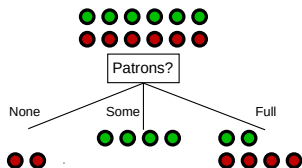
En se basant sur ce critère, qu'est-ce qui serait plus important pour classer les exemples (voir table slide suivant) et donc le plus significatif pour savoir s'il faut attendre pour une table : la présence de clients ou le type de nourriture ?

Choisir un attribut – IMPORTANCE

Ex.	Attributes									
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est
x_1	T	F	F	T	Some	\$\$\$	F	T	French	0–10
x_2	T	F	F	T	Full	\$	F	F	Thai	30–60
x_3	F	T	F	F	Some	\$	F	F	Burger	0–10
x_4	T	F	T	T	Full	\$	F	F	Thai	10–30
x_5	T	F	T	F	Full	\$\$\$	F	T	French	>60
x_6	F	T	F	T	Some	\$\$	T	T	Italian	0–10
x_7	F	T	F	F	None	\$	T	F	Burger	0–10
x_8	F	F	F	T	Some	\$\$	T	T	Thai	0–10
x_9	F	T	T	F	Full	\$	T	F	Burger	>60
x_{10}	T	T	T	T	Full	\$\$\$	F	T	Italian	10–30
x_{11}	F	F	F	F	None	\$	F	F	Thai	0–10
x_{12}	T	T	T	T	Full	\$	F	F	Burger	30–60

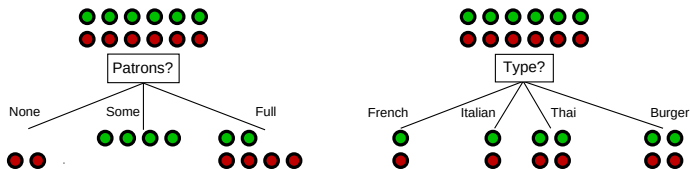
Choisir un attribut – IMPORTANCE

Idée : un “bon” attribut **sépare** les exemples en sous-ensembles qui peuvent (idéalement) être tous classés de la même façon.



Choisir un attribut – IMPORTANCE

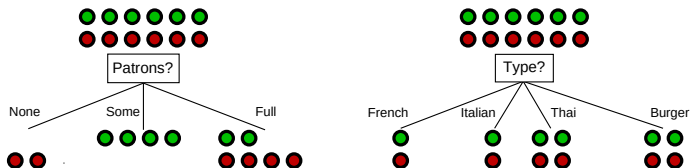
Idée : un “bon” attribut **sépare** les exemples en sous-ensembles qui peuvent (idéalement) être tous classés de la même façon.



Patrons? semble un meilleur choix : il donne de l'**information** à propos de la classification, alors que *Type?* ne permet aucune distinction entre exemples positifs et négatifs.

Choisir un attribut – IMPORTANCE

Idée : un “bon” attribut **sépare** les exemples en sous-ensembles qui peuvent (idéalement) être tous classés de la même façon.



Patrons? semble un meilleur choix : il donne de l'**information** à propos de la classification, alors que *Type?* ne permet aucune distinction entre exemples positifs et négatifs.

La **théorie de l'information** va nous permettre de formaliser cette observation.

Entropie

Intuition : plus je suis **ignorant** de la réponse à une question, plus il y aura d'**informations** contenues dans cette réponse.

En **théorie de l'information**, cette notion de “bénéfice d'information” est définie en terme d'**entropie** :

- l'entropie est une **mesure de l'incertitude d'une variable aléatoire**.
- le gain d'information correspond à une réduction de l'entropie.

Exemple

Une variable aléatoire avec seulement une seule valeur possible (par ex., une pièce dont les deux côtés sont “face”) n'entraîne aucune incertitude : son entropie est nulle. On ne gagne donc aucune information en observant sa valeur.

Entropie

L'entropie peut se mesurer en terme de **bits nécessaires pour décrire une des valeurs possibles d'une variable aléatoire équiprobable**.

Exemples :

- le résultat d'un lancer d'une pièce non truquée (ou la réponse à une question booléenne équiprobable, c-à-d dont $P(\text{true}) = P(\text{false}) = \frac{1}{2}$) peut être décrit par un seul bit
→ **1 bit d'entropie** (0 ou 1).
- le résultat du lancer d'un dé à quatre face non truqué (tétraèdre)
→ **2 bits d'entropie** (00, 01, 10 ou 11).

Entropie

L'entropie peut se mesurer en terme de **bits nécessaires pour décrire une des valeurs possibles d'une variable aléatoire équiprobable**.

Exemples :

- le résultat d'un lancer d'une pièce non truquée (ou la réponse à une question booléenne équiprobable, c-à-d dont $P(true) = P(false) = \frac{1}{2}$) peut être décrit par un seul bit
→ **1 bit d'entropie** (0 ou 1).
- le résultat du lancer d'un dé à quatre face non truqué (tétraèdre)
→ **2 bits d'entropie** (00, 01, 10 ou 11).

Et si le lancer d'une pièce donne “face” dans 99% des cas et “pile” dans 1% des cas ? L'incertitude diminue et l'entropie devrait être proche de zéro.

Entropie

Définition

L'entropie $H(V)$ d'une variable aléatoire V avec d valeurs possibles v_k (de probabilité $P(v_k)$ pour chaque k) est définie par

$$H(V) = - \sum_{k=1}^d P(v_k) \log_2 P(v_k)$$

Entropie

Définition

L'entropie $H(V)$ d'une variable aléatoire V avec d valeurs possibles v_k (de probabilité $P(v_k)$ pour chaque k) est définie par

$$H(V) = - \sum_{k=1}^d P(v_k) \log_2 P(v_k)$$

Exemples

- Si *Fair* est une pièce non truquée,

$$H(\text{Fair}) = -\left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}\right) = -(\log_2 2^{-1}) = -(-1) = 1.$$

- Si *Loaded* est une pièce truquée,

$$H(\text{Loaded}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \simeq 0.08.$$

Entropie

Utile de définir $B(q)$ comme l'entropie d'une variable aléatoire booléenne qui est vraie avec une probabilité q . Par définition de l'entropie,

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q)).$$

(Remarque : si $q = 0$ ou $q = 1$, alors $B(q) = 0$: attention à cela en implémentant B car $\log_2(0)$ produira une exception.)

Entropie

Utile de définir $B(q)$ comme l'entropie d'une **variable aléatoire booléenne** qui est vraie avec une probabilité q . Par définition de l'entropie,

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q)).$$

(**Remarque** : si $q = 0$ ou $q = 1$, alors $B(q) = 0$: attention à cela en implémentant B car $\log_2(0)$ produira une exception.)

Pour un **ensemble d'exemples** contenant p exemples positifs et n exemples négatifs, l'**entropie d'un objectif booléen** $Goal$ est naturellement définie par :

$$H(Goal) = B\left(\frac{p}{p+n}\right).$$

Entropie – choix d'un attribut

Quel attribut choisir comme racine d'un arbre de décision en construction ??

- Un test sur un attribut A va nous fournir une partie de l'entropie du training set complet E .
(par ex. pour les 12 exemples "restaurant", $p = 6$ et $n = 6$, donc $H(\text{WillWait}) = B(0.5) = 1 \rightarrow$ l'entropie de $E = 1$ bit)
- On peut mesurer cette quantité d'information en calculant l'entropie restante après avoir effectué le test sur A .
- Un test sur A (avec d valeurs distinctes possibles) va diviser E en sous-ensembles $E_1, E_2, \dots, E_d$.
- Chaque sous-ensemble E_k possède p_k exemples positifs et n_k exemples négatifs.

Entropie – choix d'un attribut

Quel attribut choisir comme racine d'un arbre de décision en construction ??

- Un test sur un attribut A va nous fournir une partie de l'entropie du training set complet E .
(par ex. pour les 12 exemples "restaurant", $p = 6$ et $n = 6$, donc $H(\text{WillWait}) = B(0.5) = 1 \rightarrow$ l'entropie de $E = 1$ bit)
- On peut mesurer cette quantité d'information en calculant l'entropie restante après avoir effectué le test sur A .
- Un test sur A (avec d valeurs distinctes possibles) va diviser E en sous-ensembles $E_1, E_2, \dots, E_d$.
- Chaque sous-ensemble E_k possède p_k exemples positifs et n_k exemples négatifs.

Entropie – choix d'un attribut

Quel attribut choisir comme racine d'un arbre de décision en construction ??

- Un test sur un attribut A va nous fournir une partie de l'entropie du training set complet E .
(par ex. pour les 12 exemples "restaurant", $p = 6$ et $n = 6$, donc $H(WillWait) = B(0.5) = 1 \rightarrow$ l'entropie de $E = 1$ bit)
- On peut mesurer cette quantité d'information en calculant l'entropie restante après avoir effectué le test sur A .
- Un test sur A (avec d valeurs distinctes possibles) va diviser E en sous-ensembles $E_1, E_2, \dots, E_d$.
- Chaque sous-ensemble E_k possède p_k exemples positifs et n_k exemples négatifs.

Entropie – choix d'un attribut

Quel attribut choisir comme racine d'un arbre de décision en construction ??

- Un test sur un attribut A va nous fournir une partie de l'entropie du training set complet E .
(par ex. pour les 12 exemples "restaurant", $p = 6$ et $n = 6$, donc $H(\text{WillWait}) = B(0.5) = 1 \rightarrow$ l'entropie de $E = 1$ bit)
- On peut mesurer cette quantité d'information en calculant l'entropie restante après avoir effectué le test sur A .
- Un test sur A (avec d valeurs distinctes possibles) va diviser E en sous-ensembles $E_1, E_2, \dots, E_d$.
- Chaque sous-ensemble E_k possède p_k exemples positifs et n_k exemples négatifs.

Entropie – choix d'un attribut

Quel attribut choisir comme racine d'un arbre de décision en construction ??

- Un test sur un attribut A va nous fournir une partie de l'entropie du training set complet E .
(par ex. pour les 12 exemples "restaurant", $p = 6$ et $n = 6$, donc $H(WillWait) = B(0.5) = 1 \rightarrow$ l'entropie de $E = 1$ bit)
- On peut mesurer cette quantité d'information en calculant l'entropie restante après avoir effectué le test sur A .
- Un test sur A (avec d valeurs distinctes possibles) va diviser E en sous-ensembles $E_1, E_2, \dots, E_d$.
- Chaque sous-ensemble E_k possède p_k exemples positifs et n_k exemples négatifs.

Entropie – choix d'un attribut

- Donc, en suivant la branche associée au choix k pour A on aura besoin de

$$B\left(\frac{p_k}{p_k + n_k}\right)$$

bits d'information pour répondre à la question.

- Or, la probabilité qu'un exemple choisi au hasard dans E possède la valeur k pour l'attribut A est $(p_k + n_k)/(p + n)$.
- Ainsi, l'entropie restante espérée après le test sur A est

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

- Le gain d'information pour A est donc

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A).$$

Entropie – choix d'un attribut

- Donc, en suivant la branche associée au choix k pour A on aura besoin de

$$B\left(\frac{p_k}{p_k + n_k}\right)$$

bits d'information pour répondre à la question.

- Or, la probabilité qu'un exemple choisi au hasard dans E possède la valeur k pour l'attribut A est $(p_k + n_k)/(p + n)$.
- Ainsi, l'entropie restante espérée après le test sur A est

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

- Le gain d'information pour A est donc

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A).$$

Entropie – choix d'un attribut

- Donc, en suivant la branche associée au choix k pour A on aura besoin de

$$B\left(\frac{p_k}{p_k + n_k}\right)$$

bits d'information pour répondre à la question.

- Or, la probabilité qu'un exemple choisi au hasard dans E possède la valeur k pour l'attribut A est $(p_k + n_k)/(p + n)$.
- Ainsi, l'entropie restante espérée après le test sur A est

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

- Le gain d'information pour A est donc

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A).$$

Entropie – choix d'un attribut

- Donc, en suivant la branche associée au choix k pour A on aura besoin de

$$B\left(\frac{p_k}{p_k + n_k}\right)$$

bits d'information pour répondre à la question.

- Or, la probabilité qu'un exemple choisi au hasard dans E possède la valeur k pour l'attribut A est $(p_k + n_k)/(p + n)$.
- Ainsi, l'entropie restante espérée après le test sur A est

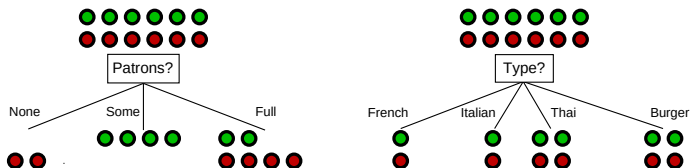
$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

- Le gain d'information pour A est donc

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A).$$

Entropie – choix d'un attribut

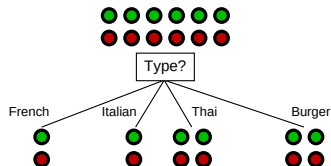
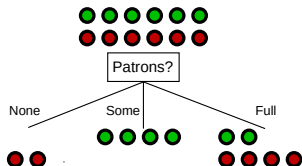
⇒ $Gain(A)$ est exactement ce qu'on a besoin pour implémenter $INFORMATION(A, \text{examples})$.



$$\begin{aligned}
 Gain(Patrons?) &= 1 - \left[\frac{2}{12} B\left(\frac{0}{2}\right) + \frac{4}{12} B\left(\frac{4}{4}\right) + \frac{6}{12} B\left(\frac{2}{6}\right) \right] \\
 &\approx 1 - \left[\frac{2}{12} \cdot 0 + \frac{4}{12} \cdot 0 + \frac{6}{12} \cdot 0.92 \right] \\
 &\approx 0.54 \text{ bits.}
 \end{aligned}$$

Entropie – choix d'un attribut

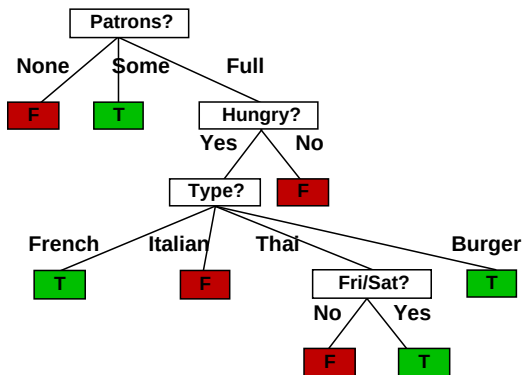
⇒ $Gain(A)$ est exactement ce qu'on a besoin pour implémenter $INFORMATION(A, \text{examples})$.



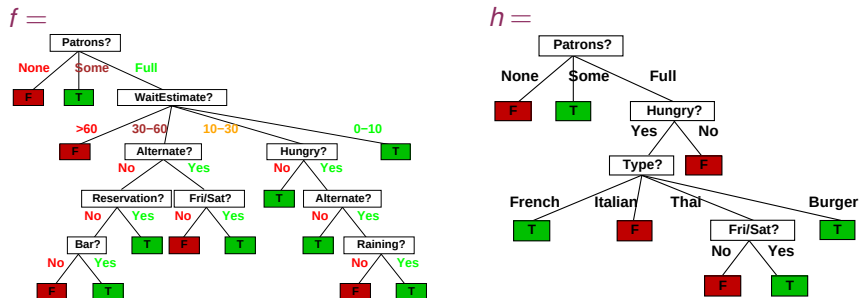
$$\begin{aligned}
 Gain(\text{Type?}) &= 1 - \left[\frac{2}{12} B\left(\frac{1}{2}\right) + \frac{2}{12} B\left(\frac{1}{2}\right) + \frac{4}{12} B\left(\frac{2}{4}\right) + \frac{4}{12} B\left(\frac{2}{4}\right) \right] \\
 &= 1 - \left[\frac{2}{12} \cdot 1 + \frac{2}{12} \cdot 1 + \frac{4}{12} \cdot 1 + \frac{4}{12} \cdot 1 \right] \\
 &= 0 \text{ bit.}
 \end{aligned}$$

Exemple d'application de l'algorithme DTL

En appliquant l'algorithme DTL sur les 12 exemples, on obtient l'hypothèse h suivante :

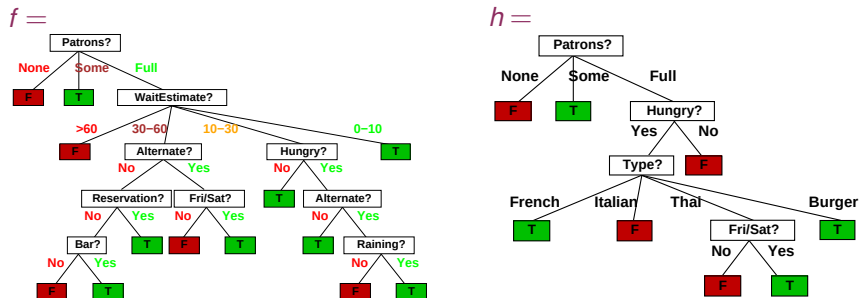


Exemple d'application de l'algorithme DTL



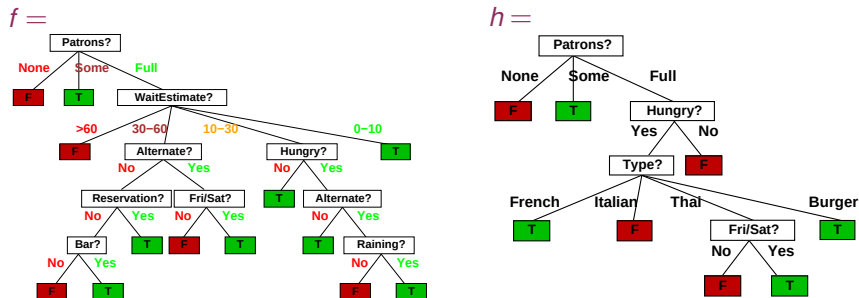
- h plus simple que f : une hypothèse plus complexe ne se justifie pas avec un si petit nombre de données.
- h consistant avec les 12 exemples mais échouera pour prédire certains cas (e.g., si on a pas faim et que le restaurant est plein, h suggère de ne pas attendre... la "réalité" (f) est plus subtile).

Exemple d'application de l'algorithme DTL



- h plus simple que f : une hypothèse plus complexe ne se justifie pas avec un si petit nombre de données.
- h consistant avec les 12 exemples mais échouera pour prédire certains cas (e.g., si on a pas faim et que le restaurant est plein, h suggère de ne pas attendre... la "réalité" (f) est plus subtile).

Exemple d'application de l'algorithme DTL



- h plus simple que f : une hypothèse plus complexe ne se justifie pas avec un si petit nombre de données.
- h consistant avec les 12 exemples mais échouera pour prédire certains cas (e.g., si on a pas faim et que le restaurant est plein, h suggère de ne pas attendre... la "réalité" (f) est plus subtile).

Evaluer et choisir la meilleure hypothèse

Mesurer la performance d'une hypothèse

Comment savoir si $h \approx f$??

- 1 Utiliser des résultats théoriques de la théorie computationnelle/statistique de l'apprentissage
- 2 Valider h sur un ensemble de nouveaux exemples (test set)

Mesurer la performance d'une hypothèse

Comment savoir si $h \approx f$??

- 1 Utiliser des résultats théoriques de la théorie computationnelle/statistique de l'apprentissage
- 2 Valider h sur un ensemble de nouveaux exemples (test set)

Mesurer la performance d'une hypothèse

Comment savoir si $h \approx f$??

- 1 Utiliser des résultats théoriques de la théorie computationnelle/statistique de l'apprentissage
- 2 Valider h sur un ensemble de nouveaux exemples (test set)

Mesurer la performance d'une hypothèse

Comment savoir si $h \approx f$??

- ❶ Utiliser des **résultats théoriques** de la théorie computationnelle/statistique de l'apprentissage
- ❷ **Valider** h sur un ensemble de nouveaux exemples (**test set**)

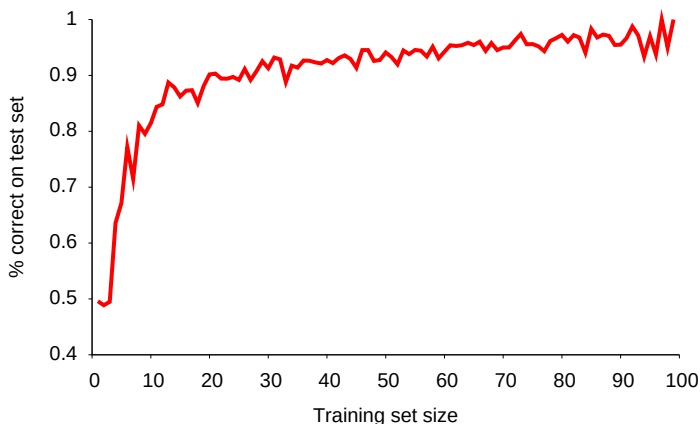
Pour un ensemble n d'exemples donnés, une **courbe d'apprentissage** montre :

- en **abscisse** : taille du training set ($x \in [1, \dots, n]$)
- en **ordonnée** : % d'exemples corrects dans le test set (dont la taille est $n - x$)

Comme l'algorithme DTL n'est pas déterministe (bris d'égalité aléatoire dans PLURALITY), un point d'une courbe d'apprentissage = moyenne de plusieurs essais.

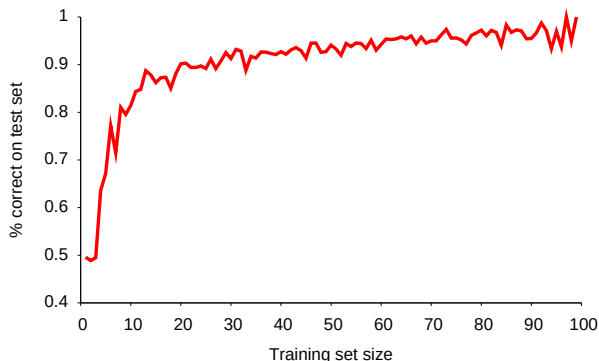
Mesurer la performance d'une hypothèse

Courbe d'apprentissage pour l'algorithme DTL.



($n = 100$, chaque point = moyenne sur 20 essais)

Mesurer la performance d'une hypothèse

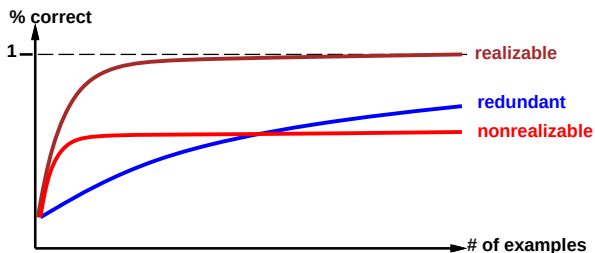


Cette courbe confirme l'intuition que plus il y a d'exemples, plus l'hypothèse h est précise.

Mesurer la performance d'une hypothèse

Une courbe d'apprentissage dépend du problème et de son espace d'hypothèses :

- est-il **réalisable** ou non ? (peut-il exprimer la fonction cible ?)
- est-il exprimé de manière fortement **redondante** ? (e.g., beaucoup d'attributs non pertinents)



Introduction aux IA génératives

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?
- Quelles sont les IA génératives que vous utilisez ?

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?
- Quelles sont les IA génératives que vous utilisez ?
- Pour quelles tâches les utilisez-vous ?

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?
- Quelles sont les IA génératives que vous utilisez ?
- Pour quelles tâches les utilisez-vous ?
- Quels sont les avantages de ces outils dans votre vie d'étudiant·e ?

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?
- Quelles sont les IA génératives que vous utilisez ?
- Pour quelles tâches les utilisez-vous ?
- Quels sont les avantages de ces outils dans votre vie d'étudiant·e ?
- Quels sont les problèmes ou les dangers de ces outils ?

Tour d'auditoire

- Quelles sont les IA génératives que vous connaissez ?
- Quelles sont les IA génératives que vous utilisez ?
- Pour quelles tâches les utilisez-vous ?
- Quels sont les avantages de ces outils dans votre vie d'étudiant·e ?
- Quels sont les problèmes ou les dangers de ces outils ?
- Comment fonctionnent-ils ?

Générateur de texte

Pour illustrer comment les IA génératives fonctionnent, nous allons créer un générateur de texte simple. Celui-ci sera extrêmement limité :

- il va construire un modèle basé sur des **trigrammes** pour générer une séquence de mots ;
- pour construire ce modèle, il va utiliser un petit **corpus de textes** tirés de la littérature classique.

Nous allons procéder par étapes. Le code est brièvement présenté en séance et disponible sur Moodle. Seuls les résultats des scripts sont inclus dans ces transparents.

Etape 1 : création du corpus de livres

Le projet [Gutenberg](http://www.gutenberg.org) (www.gutenberg.org) héberge des milliers de livres libres de droits, qui peuvent être téléchargés sous format `txt`.

Nous allons télécharger quelques livres pour créer un [corpus](#) de textes en français, qui nous servira à construire notre modèle.

Notre premier script va décrire ce corpus en ouvrant chaque fichier pour en extraire des informations (titre, auteur, nombre de lignes de texte).

Etape 1 : création du corpus de livres

Exécution de : `python3 step1.py`

```
Fichiers :  
  books/candide.txt  
  books/fleur_du_mal.txt  
  books/meaulnes.txt  
  books/miserables.txt  
  books/zarathoustra.txt  
Informations :  
  Candide, ou l'optimisme  
    -- Voltaire (3920 lignes)  
  Les Fleurs du Mal  
    -- Charles Baudelaire (4947 lignes)  
  Le Grand Meaulnes  
    -- Alain-Fournier (8098 lignes)  
  Les misérables Tome I: Fantine  
    -- Victor Hugo (13895 lignes)  
  Ainsi Parlait Zarathoustra  
    -- Friedrich Wilhelm Nietzsche (15238 lignes)
```

Etape 2 : obtenir les textes

La deuxième étape consiste à extraire le texte brut de chaque livre, en supprimant les autres données (informations, licence, etc.).

Pour tester cela, notre deuxième script va extraire les x premières lignes d'un livre donné.

Etape 2 : obtenir les textes

Exécution de : `python3 step2.py books/miserables.txt 20`

Chapitre I

Monsieur Myriel

En 1815, M. Charles-François-Bienvenu Myriel était évêque de Digne. C'était un vieillard d'environ soixante-quinze ans; il occupait le siège de Digne depuis 1806.

Quoique ce détail ne touche en aucune manière au fond même de ce que nous avons à raconter, il n'est peut-être pas inutile, ne fût-ce que pour être exact en tout, d'indiquer ici les bruits et les propos qui avaient couru sur son compte au moment où il était arrivé dans le diocèse. Vrai ou faux, ce qu'on dit des hommes tient souvent autant de place dans leur vie et surtout dans leur destinée que ce qu'ils font. M. Myriel était fils d'un conseiller au parlement d'Aix; noblesse de robe. On contait de lui que son père, le réservant pour hériter de sa charge, l'avait marié de fort bonne heure, à dix-huit ou vingt ans, suivant un usage assez répandu dans les familles parlementaires. Charles Myriel, nonobstant ce mariage, avait, disait-on, beaucoup fait parler de lui. Il

Etape 3 : mots uniques

Lors de cette troisième étape, nous allons

- lire l'ensemble des textes, ligne par ligne,
- séparer (grâce aux espaces) les lignes pour obtenir les mots,
- les ajouter dans un `set` qui contiendra tous les mots uniques,
- afficher le nombre de mots uniques,
- afficher les x plus longs mots uniques.

Etape 3 : mots uniques

Exécution de : `python3 step3.py 10`

```
Unique words: 48607
Longest words:
  Pangloss?--Qu'entends-je?
  Saint-Jacques-du-Haut-Pas.
  cuisinière.--Mademoiselle,
  Thunder-ten-tronckh?--Oui,
  moi-même?.....
  folliculaire.--Qu'appellez-vous
  d'extraordinaire.--Croyez-vous,
  _Waldberghoff-trarbk-dikdorff._
  Suppositions--Désespoir--Fatigue--Je
  métaphysico-théologo-cosmolonigologie.
```

Etapes 4 à 6 : gestion des caractères spéciaux

Le script précédent montre des problèmes :

- un double tiret est utilisé quand un des personnages parle et il n'y a pas toujours d'espaces autour des tirets quand cela se produit. Nous allons remplacer les doubles tirets par des espaces (avant la séparation des lignes en mots).
- il faudrait retirer les caractères de ponctuation qui se trouvent devant ou après un mot (mais les laisser à l'intérieur des mots pour conserver des mots comme "aujourd'hui"),
- il faudrait mettre en minuscules tous les mots pour éviter de compter deux fois les mêmes mots si une majuscule est utilisée.

Etape 4 : gestion des doubles tirets

Exécution de : `python3 step4.py 10`

```
Unique words: 47858
Longest words:
  Guy-Étienne-Alexandre,
  Saint-Germain-des-Prés
  adjudant-garde-chiourme
  chapeau-capote-capeline
  d'Ailly-le-Haut-Clocher,
  Charles-François-Bienvenu
  Saint-Jacques-du-Haut-Pas.
  moi-même?.....
  _Waldberghoff-trarbk-dikdorff._
  métaphysico-théologo-cosmolonigologie.
```

Etape 5 : liste des caractères de ponctuation

Nous créons d'abord une fonction qui va détecter tous les caractères de ponctuation, grâce au package `unicodedata`.

Exécution de : `python3 step5.py`

```
"%/,? [!>? (] :_)?'##??-?.*?«
```

Etape 6 : gestion des caractères de ponctuation et des majuscules

Exécution de : `python3 step6.py 10`

```
Unique words: 26165
Longest words:
l'heautontimoroumenos
s'entretiendraient-ils
saint-germain-des-prés
chapeau-capote-capeline
adjudant-garde-chiourme
d'ailly-le-haut-clocher
charles-françois-bienvenu
saint-jacques-du-haut-pas
waldberghoff-trarbk-dikdorff
métaphysico-théologo-cosmolonigologie
```

Observez comme le nombre de mots uniques a diminué.

Etape 7 : obtenir les mots les plus fréquents

Notre futur générateur de texte ne va pas utiliser cette information, mais il est intéressant à ce stade de voir quels sont les mots les plus fréquents dans notre corpus.

Notre prochain script va construire un dictionnaire pour compter la fréquence des mots et afficher les x mots les plus fréquents.

Etape 7 : obtenir les mots les plus fréquents

Exécution de : `python3 step7.py 10`

Most frequent words:

13621	de
10399	et
9062	la
7647	le
5817	à
5364	les
5169	il
4579	que
4267	qui
4025	un

Etape 8 : obtenir les bigrammes les plus fréquents

Un **bigramme** est une paire de mots consécutifs.

Le prochain script va construire un dictionnaire pour compter la fréquence des bigrammes et afficher les x bigrammes les plus fréquents.

Etape 8 : obtenir les bigrammes les plus fréquents

Exécution de : `python3 step8.py 10`

```
Most frequent bigrams:
1768      ('de', 'la')
732       ('à', 'la')
640       ('et', 'de')
596       ('dans', 'la')
566       ('il', 'y')
522       ('que', 'je')
443       ('dans', 'le')
419       ('ce', 'qui')
413       ('je', 'ne')
393       ('et', 'la')
```

Etape 9 : successeurs possibles d'un mot

Un bigramme en soit ne nous aide pas encore beaucoup. Nous allons les utiliser pour construire un dictionnaire qui va nous permettre de connaître les mots qui pourraient suivre un mot donné. Ce dictionnaire associe à chaque mot (clef) une liste de mots qui les ont suivis dans le corpus.

Notre prochain script affiche la liste des successeurs rencontrés pour un mot donné.

Etape 9 : successeurs possibles d'un mot

Exécution de : `python3 step9.py prince`

```
['souverain', 'aussi', "l'invita", 'de', 'qui', 'dont',  
'de', 'de', 'des', 'aussi', 'est', 'de', 'harassé',  
'de', 'de', "qu'important", 'celui']
```

Remarquez que le mot “de” est fréquent dans cette liste. Cela est dû au fait que le mot “de” a suivi 6 fois le mot “prince” dans notre corpus.

Etape 10 : première version du générateur

Un premier générateur de texte pourrait :

- partir du dernier mot d'une phrase incomplète ;
- la continuer en choisissant un mot parmi ceux qui ont suivi ce mot dans le corpus, en favorisant les mots successeurs les plus fréquents ;
- réitérer le processus à partir du dernier mot généré, jusqu'à obtenir une "phrase" de longueur donnée.

C'est exactement ce que va faire le prochain script, à partir d'une chaîne qui constitue un début de phrase.

Etape 10 : première version du générateur

Exécution de :

```
python3 step10.py "Il était une fois, un prince"
```

```
Il était une fois, un prince de l'autre jour accepter ce qu'on trouve  
un galérien de la place au granit elle habitait le brûlerez alors il  
entrevoit de ton bétail il.
```

Le modèle est bien sûr trop simpliste. Il ne prend pas en compte la syntaxe, la sémantique, etc. et ce qu'il produit n'a aucun sens.

Etape 11 : obtenir les trigrammes

Pour améliorer notre modèle, nous allons utiliser des **trigrammes** : des triplets de mots consécutifs.

Le prochain script va construire un dictionnaire qui va nous permettre de connaître les mots qui pourraient suivre un couple de mots donné. Ce dictionnaire associe à chaque couple de mots (clef) une liste de mots qui les ont suivis dans le corpus.

Etape 11 : obtenir les trigrammes

Exécution de : `python3 step11.py mange des`

After the pair (mange, des) possible successors are:
['biscuits']

Exécution de : `python3 step11.py zarathoustra est`

After the pair (zarathoustra, est) possible successors are:
['indulgent', 'tenté', 'un', 'tout', 'impie', 'le', 'heureux']

Etape 12 : deuxième version du générateur

Un deuxième générateur de texte pourrait :

- partir des deux derniers mots d'une phrase incomplète ;
- la continuer en choisissant un mot parmi ceux qui ont suivi cette paire de mots dans le corpus ;
- réitérer le processus à partir des deux derniers mots générés, jusqu'à obtenir une "phrase" de longueur donnée.

Etape 12 : nouvelle version du générateur

Exécution de :

```
python3 step12.py "la france"
```

La france était menacée j'ai offert ma poitrine je n'étais pas
un château dans le creux muet des vieux antres païens
t'appellent au secours de leurs infortunes un profond silence régnait
sur ce roseau inébranlable toucher de ses prunelles pâles clairs
fanaux vivantes opales qui me rafraîchissaient le front écorché
au-dessus de leurs compagnons s'étaient enfuis à l'approche de
quelque chose

Etape 12 : nouvelle version du générateur

Exécution de :

```
python3 step12.py "en arrivant à l'UMONS ce matin, j'ai vu"
```

```
En arrivant à l'umons ce matin, j'ai vu de quelle nature était  
l'obéissance des deux bras écartés veux-tu que je pus distinguer  
un grand fichu de laine et leurs infortunes un profond respect  
et de la capitale il est dangereux d'être héritier nous luttons  
encore pied à chaque minute sur le mur je demandai à retourner  
ensuite la charrue on voyait presque jusqu'au fond mais c'est ainsi
```

Générateur de texte

Notre générateur commence à produire des listes de mots qui ressemblent à des phrases³, mais elles n'ont toujours pas de sens.

Cependant l'idée principale est là : **construire un modèle, à partir d'un corpus, qui permet de “prédire” le mot suivant en fonction des mots précédents.**

Comme on va le voir, les performances des véritables IA génératives sont dûes notamment au fait que

- les modèles utilisés sont beaucoup plus complexes que nos trigrammes et ;
- les corpus de données pour construire ces modèles sont gigantesques.

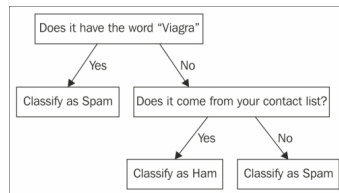
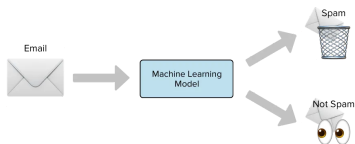
3. On pourrait encore améliorer les choses, par exemple en calculant les mots qui terminent souvent une phrase, pour séparer le texte en phrases courtes, etc.

Introduction aux LLM

Pour comprendre le fonctionnement des IA génératives, nous allons brièvement aborder les notions suivantes :

- Apprentissage profond (Deep Learning)
- Langages naturels & IA
- Grands modèles de langage (LLM)

Modèles

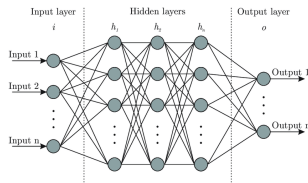


Arbres de décisions $\Rightarrow$ (trop) simples mais lisibles par l'humain

Images : Raúl Garreta, Guillermo Moncecchi (Learning scikit-learn : Machine Learning in Python)

Réseaux de neurones

Les **réseaux de neurones** forment un autre type de modèles.



⇒ complexe et puissant mais illisible pour l'humain

Deep learning = plusieurs couches de réseaux de neurones

Image gauche : <http://www.extremetech.com/wp-content/uploads/2015/07/neural-net-head.jpg>

Image droite : Bre, Gimenez et Fachinotti (2017)

Modèle de langage

Un **modèle de langage** est un modèle ou algorithme permettant de prédire les mots qui vont suivre, de façon probabiliste :

En arrivant à l'UMONS ce matin, j'ai vu...

⇒ Etant donné une séquence de mots $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, estimer la distribution de probabilité pour le mot suivant $x^{(t+1)}$:

$$P(x^{(t+1)} | x^{(1)}, \dots, x^{(t)}),$$

où $x^{(t+1)}$ peut être tout mot compris dans un ensemble de mots $V = \{w_1, \dots, w_N\}$ appelé le **vocabulaire**.

Modèle de langage

On ne peut pas se limiter aux mots du dictionnaire.

Le **vocabulaire** doit être étendu à toutes formes possibles (conjugaisons, pluriel, genre, etc.).

Cela donne un vocabulaire contenant des centaines de milliers de mots (démultiplié par le nombre de langues que le modèle peut accepter).

De plus, pour construire un modèle qui utilise des réseaux de neurones, il faut en spécifier (apprendre) les **paramètres**.

Le nombre de paramètres dans les modèles récents se compte en milliards.

Grands modèles de langage (LLM)

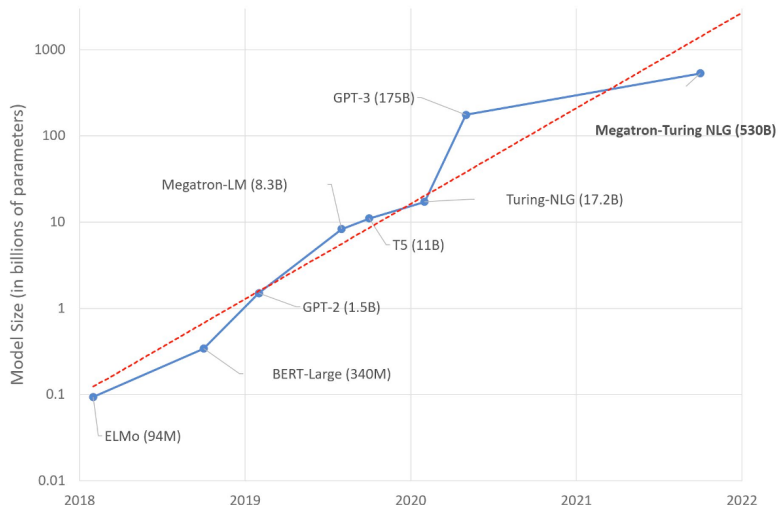


Image : Microsoft

Conseils d'utilisation des IA génératives

- Soyez prudent·e. Si vous ne comprenez pas ce que produit l'IA, vous passez à côté de l'objectif d'apprentissage et vous risquez d'être sanctionné·e ;
- N'oubliez pas qu'à l'examen, vous serez seul·e face à votre copie ou votre enseignant.
- Une utilisation intéressante dans le cadre de vos études : utiliser l'IA pour comprendre des éléments précis, en demandant de générer des exercices, de corriger et d'expliquer vos solutions,

Conseils d'utilisation des IA génératives

- La qualité des réponses dépend de la qualité des prompts. Il est difficile de faire des prompts de qualité sur un sujet précis, si on ne le comprend pas.
- Ne prenez jamais une réponse comme parfaite. Soyez critique et remettez la en question. Vous avez maintenant une idée de comment fonctionnent ces IA génératives. Il s'agit uniquement d'un modèle prédictif basé sur des données pré-existantes.
- Le travail cognitif que vous effectuez pour composer un prompt et interpréter la réponse peut être aussi précieux que le code généré par l'outil.

Limites, dangers et conséquences

Si vous utilisez un outil puissant, comme une tronçonneuse par exemple, il vaut mieux connaître les dangers, les limites et les conséquences de son utilisation.

Nous allons aborder quelques “mauvais” exemples et pointer quelques articles qui mettent en lumière les limites ou les conséquences de l'utilisation des IA génératives.

Exemples de réponses erronées

Les IA peuvent produire des réponses erronées.

- Dans un mémoire en informatique soumis à l'UMONS un peu après l'apparition de ChatGPT, un rapporteur du mémoire a été très étonné de lire une affirmation sur un résultat théorique qui était faux. Cette affirmation était justifiée par une “référence scientifique” totalement inventée et inexistante.
- Article de *Pour la Science* (8 juillet 2023) qui montre quelques “élucubrations mathématiques” de ChatGPT :

<https://www.pourlascience.fr/p/opinions/les-elucubrations-mathematiques-de-chatgpt-25434.php#Echobox=1740996109>

(L'article date de juillet 2023 : ChatGPT se comporte mieux depuis sur certains de ces exemples.)

Exemples de réponses erronées

Lors d'un TP du cours d'*Algorithmique et programmation impérative*, un étudiant a demandé à ChatGPT de résoudre un des premiers exercices, qui est extrêmement simple.

La même erreur, à la même question, a pu être reproduite le 24 mars 2025.

Soit $a=1$, $b=2$, $c=3$ et $d=4$. Donne moi un code Python qui inverse l'ordre des valeurs des quatre variables. Tu peux utiliser une seule variable temporaire et uniquement des simples assignations.

Exemples de réponses erronées

Voici un code Python qui inverse l'ordre des valeurs des quatre variables `a`, `b`, `c` et `d` en utilisant une seule variable temporaire et uniquement des simples assignations :

python

 Copy

 Edit

```
# Initialisation des variables
```

```
a = 1
```

```
b = 2
```

```
c = 3
```

```
d = 4
```

```
# Inversion des valeurs en utilisant une seule variable temporaire
```

```
temp = a
```

```
a = d
```

```
d = c
```

```
c = b
```

```
b = temp
```

```
# Affichage des résultats
```

```
print("a =", a)
```

```
print("b =", b)
```

```
print("c =", c)
```

```
print("d =", d)
```

Exemples de réponses erronées

Résultat

a = 4

b = 1

c = 2

d = 3

Exemples de réponses erronées

En soumettant une question d'un examen d'IA de janvier 2025 à ChatGPT, ce dernier donnait des réponses erronées de manière répétée.

La conversation a été réalisée le 24 mars 2025 et les réponses de ChatGPT étant longues, voici quelques extraits de cet échange.

Soit l'ensemble de clauses suivantes en logique propositionnelle
:

C1: $\neg S1,1$

C2: $S1,2$

(. . .)

C17: $\neg W2,2 \vee \neg W3,1$

Prouve moi par contradiction que $W2,2$ est vrai, en ne donnant que l'ensemble des applications de la règle de résolution qui sont utiles.

La règle de résolution s'applique sur deux clauses qui contiennent un littéral et son complémentaire.



Pour prouver par contradiction que $W2, 2$ est vrai en utilisant la règle de résolution, nous allons supposer que $W2, 2$ est faux, c'est-à-dire $\neg W2, 2$, et essayer d'en déduire une contradiction.

Hypothèse de départ : $\neg W2, 2$ est vrai.

Nous allons maintenant appliquer la règle de résolution sur les clauses contenant $W2, 2$ et $\neg W2, 2$.

Étapes de résolution :

1. **Clause C10 :** $\neg W2, 2 \vee S1, 2$

Résolution avec $\neg W2, 2$:

Nous obtenons $S1, 2$.

2. **Clause C7 :** $\neg S1, 2 \vee W1, 1 \vee W1, 3 \vee W2, 2$

Nous avons $S1, 2$ de l'étape précédente, donc on résout avec $\neg S1, 2$ pour obtenir :

$W1, 1 \vee W1, 3 \vee W2, 2$.

(...)

Je t'ai demandé d'appliquer la règle de résolution sur DEUX clauses dont l'une contient un littéral (par exemple A) et l'autre son complémentaire (par exemple not A). La preuve s'arrêtera quand tu produiras une clause vide (car cela correspondra à une contradiction).

Merci pour la précision ! Effectivement, la règle de résolution s'applique sur deux clauses dont l'une contient un littéral et l'autre son complémentaire, et la preuve se termine lorsque l'on obtient une clause vide, ce qui indique une contradiction.

Je vais maintenant appliquer la règle de résolution pour prouver par contradiction que $W2, 2$ est vrai, en appliquant la règle de résolution sur les clauses pertinentes jusqu'à obtenir une clause vide.

Hypothèse de départ : $\neg W_{2,2}$ est vrai.

Nous allons résoudre les clauses contenant $W_{2,2}$ et $\neg W_{2,2}$.

Étapes de résolution :

1. **Clause C10 :** $\neg W_{2,2} \vee S_{1,2}$

Clause C14 : $\neg W_{2,2} \vee S_{2,1}$

Résolution entre $\neg W_{2,2}$ (présente dans les deux clauses) : Nous obtenons :

$$S_{1,2} \vee S_{2,1}$$

(...)

A ta première étape, tu utilises deux clauses qui ont le même littéral (not $W_{2,2}$), alors que je t'ai demandé de l'appliquer avec un littéral dans une des deux clauses, et dans l'autre son complémentaire. Donc ici tu aurais pu par exemple appliquer C2 et C7 puisque C2 contient $S_{1,2}$ et C7 contient not $S_{1,2}$

Tu as tout à fait raison, je me suis trompé dans l'application de la règle de résolution. Je vais corriger cela en appliquant correctement la règle de résolution sur des clauses contenant un littéral et son complémentaire, comme tu l'as demandé.

Repartons de l'hypothèse $\neg W2, 2$ et appliquons la règle de résolution sur les bonnes clauses jusqu'à obtenir une contradiction.

Hypothèse de départ : $\neg W2, 2$ est vrai.

Étapes de résolution :

1. **Clause C10 :** $\neg W2, 2 \vee S1, 2$

Clause C7 : $\neg S1, 2 \vee W1, 1 \vee W1, 3 \vee W2, 2$

Résolution entre $S1, 2$ (présente dans C7) et $\neg S1, 2$ (présente dans C10) :

$$W1, 1 \vee W1, 3 \vee W2, 2$$

2. **Clause C8 :** $\neg W1, 1 \vee S1, 2$

Nous avons $S1, 2$ de l'étape précédente, donc on résout avec $\neg W1, 1$ pour obtenir :

$$W1, 3 \vee W2, 2$$

(...)

Ta deuxième étape est fautive : tu supprimes $W_{1,1}$ de ce que tu as obtenu à l'étape précédente en utilisant C8, mais tu oublies de rajouter $S_{1,2}$

Tu as raison, je me suis trompé à la deuxième étape. Je vais maintenant appliquer correctement la règle de résolution en prenant bien en compte tous les éléments nécessaires. Merci de ta patience !

Repartons de l'hypothèse $\neg W_{2,2}$ et appliquons la règle de résolution correctement.

Hypothèse de départ : $\neg W2, 2$ est vrai.

Étapes de résolution :

1. **Clause C10 :** $\neg W2, 2 \vee S1, 2$

Clause C7 : $\neg S1, 2 \vee W1, 1 \vee W1, 3 \vee W2, 2$

Résolution entre $\neg W2, 2$ et $\neg W2, 2$ nous donne :

Nous obtenons $S1, 2$ à partir de C10.

2. **Clause C7 :** $\neg S1, 2 \vee W1, 1 \vee W1, 3 \vee W2, 2$

Nous avons $S1, 2$, donc on résout avec $\neg S1, 2$ dans C7. Cela donne :

$$W1, 1 \vee W1, 3 \vee W2, 2$$

(...)

Tu refais le même type d'erreur, cette fois-ci dès la première étape.

Merci pour ta patience et pour m'avoir fait remarquer l'erreur. Tu as raison, je vais maintenant procéder de manière plus précise en appliquant correctement la règle de résolution dès la première étape.

Je vais reprendre depuis le début en suivant la bonne procédure pour appliquer la règle de résolution sur des clauses contenant un littéral et son complémentaire. Voici la solution corrigée.

Hypothèse de départ : $\neg W_{2,2}$ est vrai.

Étapes de résolution :

1. **Clause C10 :** $\neg W_{2,2} \vee S_{1,2}$

Clause C7 : $\neg S_{1,2} \vee W_{1,1} \vee W_{1,3} \vee W_{2,2}$

Résolution entre $\neg W_{2,2}$ (dans C10) et $\neg W_{2,2}$ (dans C7) nous donne $S_{1,2}$.

Et les erreurs se répètent...

Exemples de réponses erronées

A la demande de trouver un nom pour un logiciel destiné à aider les chimistes via une vision polyédrale de certains de leurs problèmes, ChatGPT a donné plusieurs propositions. La cinquième, où il fait de l'humour, montre qu'il confond Maître Gims et Stromae.

5. **Chimieoutai** – parce que ton polygone a disparu comme dans la chanson de Maître Gims. 🎵

Exemples de réponses erronées

Un autre type de réponse erronée survient quand une IA ne respecte par exemple pas les règles imposées.

- Article du MIT (5 mars 2025) qui montre comment une IA triche à un jeu d'échecs sans qu'on le lui ait demandé :

www.technologyreview.com/2025/03/05/1112845/

[the-download-ai-can-cheat-at-chess-and-the-future-of-search/](#)

- Article de la RTBF (9 mars 2025) qui traite de ce sujet :

www.rtbef.be/article/

[les-intelligences-artificielles-trichent-pour-gagner-aux-echecs-et-les-expert](#)

Ethique et libertés fondamentales

Les IA génératives posent des problèmes éthiques et de respect des libertés fondamentales. Parmi ceux-ci :

- Problèmes des biais (induits par les données d'apprentissage, pensez par exemple à notre corpus de textes classiques qui oriente le type de mots générés) ;
- Respect de la vie privée (données collectées par des entreprises privées basées dans des pays qui n'appliquent pas la législation européenne) ;
- Confidentialité (le FNRS interdit par exemple aux experts qui doivent évaluer des projets de les soumettre à des IA génératives) ;
- Respect des droits d'auteur ;
- Pertes d'emploi ;
- etc.

Ethique et libertés fondamentales

L'Europe s'est dotée en février 2025 d'une législation pour des systèmes d'IA sûrs, éthiques et dignes de confiance (AI act).

- Page web du conseil européen :

`www.consilium.europa.eu/fr/policies/artificial-intelligence`

- Article RTBF du 2 février 2025 :

`www.rtbf.be/article/`

`la-toute-premiere-legislation-generale-au-monde-sur-l-ia-est-europeenne-et-e`

Mais la plupart des IA génératives à disposition ne sont pas européennes... est-ce que cet AI act sera facile à faire respecter ?

Quid du contexte politique des pays qui les hébergent ?

Impact sur la pensée critique

Une étude de *Microsoft Research* (2025) montre que l'utilisation d'IA génératives (GenAI) peut avoir une évolution sur la pensée critique.

www.microsoft.com/en-us/research/wp-content/uploads/2025/01/lee_2025_ai_critical_thinking_survey.pdf

Résumé de leurs conclusions :

- L'utilisation des outils GenAI modifie l'effort de la pensée critique de trois manières :
 - ▶ mémorisation et compréhension : le focus passe de la collecte d'informations à la vérification des informations.
 - ▶ application : l'accent se déplace de la résolution de problèmes à l'intégration des réponses générées par l'IA.
 - ▶ analyse, synthèse et évaluation : l'effort passe de l'exécution de la tâche à la gestion de la tâche.

Impact sur la pensée critique

- L'utilisation de GenAI crée de nouvelles tâches cognitives : intégration des réponses, évaluation du contenu généré par l'IA.
- Certaines tâches cognitives deviennent moins nécessaires comme la collecte d'informations.
- Certaines tâches cognitives restent nécessaires, mais ont évolué, comme la vérification de l'information (sources externes et propre expertise).

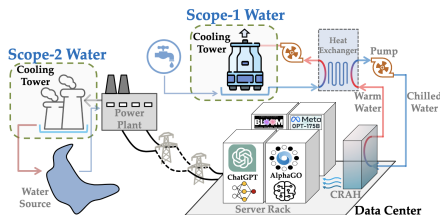
Impact sur la pensée critique

- Les travailleurs passent de l'exécution des tâches à la supervision où ils doivent guider et surveiller l'IA pour produire des résultats de qualité.
- La responsabilité du travail reste du ressort de l'utilisateur humain, même si la production matérielle est déléguée à l'outil GenAI.
- Il est important de maintenir des compétences de base en collecte d'informations et en résolution de problèmes pour éviter la dépendance excessive à l'IA.

Impact écologique

Les ressources nécessaires pour entraîner et utiliser les modèles de langage ont un impact écologique non négligeable (eau pour refroidissement des serveurs, électricité, etc.).

D'après des chercheurs de l'UC Riverside et de l'UT Arlington (USA), entraîner le modèle GPT-3 a évaporé 700000 litres d'eau potable.



Source et image : Li, P., Yang, J., Islam, M. A., Ren, S. (2022). Making AI Less Thirsty ? : Uncovering and Addressing the Secret Water Footprint of AI Models. Artificial Intelligence (AI), 4.

Impact écologique

“L’augmentation des besoins en calcul informatique pour l’IA a conduit à une hausse significative des émissions de gaz à effet de serre. Par exemple, Google a constaté une augmentation de 13% de ses émissions en un an, portant l’augmentation totale à 48% sur cinq ans.”

Source : Ministères Aménagement du territoire, Transition écologique, France, 10 mars 2025

Impact écologique

[rtbf.be](https://www.rtb.be)

L'IA va-t-elle ruiner nos efforts climatiques ? - RTBF Actus

Par @sarah-poucet-158

2-3 minutes

Benoît Frénay pointe le large spectre que regroupent les intelligences artificielles. Leur consommation est très variable en fonction de leur taille ou de leur technologie. Certaines IA peuvent ainsi fonctionner sur un PC individuel quand GPT-4, le dernier modèle d'OpenAI encore à l'étude, demande l'équivalent de deux mois de consommation énergétique d'un ménage américain moyen pour effectuer une seule tâche.

Ceci dit, une IA plus légère pourrait aussi devenir problématique. "Le paradoxe de Jevons nous dit qu'en fait, parfois, réduire l'impact énergétique d'une technologie va faire qu'on l'utilise plus et du coup globalement augmenter son impact" alerte Benoît Frénay. Ainsi, si seulement 1000 personnes utilisent OpenAI GPT-4 mais qu'un million d'utilisateurs se connectent chaque jour sur Chat GPT, Chat GPT a beau fonctionner de façon plus légère, son impact environnemental sera quand même plus grand.

Extrait d'un article de la RTBF⁴ sur l'impact écologique (12 janvier 2025).

4. www.rtb.be/article/1-lia-va-t-elle-ruiner-nos-efforts-climatiques-11487611

Résumé du chapitre

Résumé du chapitre

Ce chapitre est une très brève introduction à l'apprentissage automatique ([Machine Learning](#))

- l'apprentissage [nécessaire](#) pour des environnements inconnus et pour certains problèmes dont on ne connaît pas de solution algorithmique
- un agent qui apprend = agent de performance + éléments d'apprentissage
- les méthodes d'apprentissage dépendent du [feedback](#) disponible, du [type de composant](#) à améliorer et de sa [représentation](#)

Résumé du chapitre

- le but de l'apprentissage supervisé est de trouver une hypothèse simple et consistante avec les exemples du training set
- l'apprentissage d'arbres de décision utilise la notion de gain d'information
- la performance de l'apprentissage = mesure de la précision des prédictions sur un ensemble test de nouveaux exemples